UVM Session 3

Passive/Active Agent

```
class shift_reg_config extends uvm_object;
  `uvm_object_utils(shift_reg_config)

  virtual shift_reg_if shift_reg_vif;
  uvm_active_passive_enum is_active;

  function new(string name = "shift_reg_config");
    super.new(name);
  endfunction
endclass
```

```
class shift_reg_test extends uvm_test;
  `uvm_component_utils(shift_reg_test)

  shift_reg_env env;
  shift_reg_config shift_reg_cfg;
  shift_reg_main_sequence main_seq;
  shift_reg_reset_sequence reset_seq;

  function new(string name = "shift_reg_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = shift_reg_env::type_id::create("env",this);
    shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg",this);
    main_seq = shift_reg_main_sequence::type_id::create("main_seq",this);
    reset_seq = shift_reg_reset_sequence::type_id::create("reset_seq",this);

    if (!uvm_config_db #(virtual shift_reg_if)::get(this, "","shift_reg_IF", shift_reg_cfg.shift_reg_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift_reg from the uvm_config_db");

    shift_reg_cfg.is_active = UVM_ACTIVE;
    uvm_config_db #(shift_reg_config)::set(this, "*", "CFG", shift_reg_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    uvm_top.print_topology();
    factory.print();
    //reset sequence
    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)

    //main sequence
    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
    phase.drop_objection(this);
  endtask
endclass: shift_reg_test
```

```
class shift_reg_agent extends uvm_agent;
  `uvm_component_utils(shift_reg_agent)
  shift_reg_sequencer sqr;
  shift_reg_driver drv;
  shift_reg_monitor mon;
  shift_reg_config shift_reg_cfg;
  uvm_analysis_port #(shift_reg_seq_item) agt_ap;

  function new (string name = "shift_reg_agent", uvm_component parent = null);
   super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG", shift_reg_cfg))begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end

    if (shift_reg_cfg.is_active == UVM_ACTIVE) begin
      sqr = shift_reg_sequencer::type_id::create("sqr",this);
      drv = shift_reg_driver::type_id::create("drv",this);
    end
    mon = shift_reg_monitor::type_id::create("mon",this);
    agt_ap = new("agt_ap", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    if (shift_reg_cfg.is_active == UVM_ACTIVE) begin
      drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
      drv.seq_item_port.connect(sqr.seq_item_export);
    end
    mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
    mon.mon_ap.connect(agt_ap);
  endfunction
endclass
```

Factory Override

```
class shift_reg_seq_item extends uvm_sequence_item;
 `uvm_object_utils(shift_reg_seq_item)

  rand bit reset, serial_in;
  rand direction_e direction;
  rand mode_e mode;
  rand bit [5:0] datain;
  logic [5:0] dataout;

 function new(string name = "shift_reg_seq_item");
  super.new(name);
 endfunction

 function string convert2string();
  return $sformatf("%s reset = 0b%b, serial_in = 0b%b, direction = %s,
 endfunction

 function string convert2string_stimulus();
  return $sformatf("reset = 0b%b, serial_in = 0b%b, direction = %s, mo
 endfunction


constraint rst_con {
 reset dist {1:= 2, 0:= 98};
}

endclass

class shift_reg_seq_item_disable_rst extends shift_reg_seq_item;
 `uvm_object_utils(shift_reg_seq_item_disable_rst)

 function new(string name = "shift_reg_seq_item_disable_rst");
  super.new(name);
 endfunction

constraint rst_con {reset == 0;}
endclass
```

```
class shift_reg_test extends uvm_test;
  `uvm_component_utils(shift_reg_test)

  shift_reg_env env;
  shift_reg_config shift_reg_cfg;
  shift_reg_main_sequence main_seq;
  shift_reg_reset_sequence reset_seq;

  function new(string name = "shift_reg_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = shift_reg_env::type_id::create("env",this);
    shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg",this);
    main_seq = shift_reg_main_sequence::type_id::create("main_seq",this);
    reset_seq = shift_reg_reset_sequence::type_id::create("reset_seq",this);
    set_type_override_by_type(shift_reg_seq_item::get_type(), shift_reg_seq_item_disable_rst::get_type());

    if (!uvm_config_db #(virtual shift_reg_if)::get(this, "","shift_reg_IF", shift_reg_cfg.shift_reg_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift_reg from the uvm_config_db");

    shift_reg_cfg.is_active = UVM_ACTIVE;
    uvm_config_db #(shift_reg_config)::set(this, "*", "CFG", shift_reg_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    uvm_top.print_topology();
    factory.print();
    //reset sequence
    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)

    //main sequence
    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
    phase.drop_objection(this);
  endtask
endclass: shift_reg_test
```

3 Env in one test

```
class top_test extends uvm_test;
  `uvm_component_utils(top_test)

  virtual DUT1_if DUT1_if_test; // DUT1
  virtual DUT2_if DUT2_if_test; // DUT2
  virtual DUT3_if DUT3_if_test; // DUT3

  DUT1_config my_DUT1_config; // DUT1
  DUT2_config my_DUT2_config; // DUT2
  DUT3_config my_DUT3_config; // DUT3

  main_sequence seq;

  function new(string name = "top_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    seq = main_sequence::type_id::create("seq");
    env_DUT1 = DUT1_env::type_id::create("env_DUT1",this);
    env_DUT2 = DUT2_env::type_id::create("env_DUT2",this);
    env_DUT3 = DUT3_env::type_id::create("env_DUT3",this);

    if (!uvm_config_db #(virtual DUT1_if)::get(this, "","DUT1_IF", my_DUT1_config.DUT1_if))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the DUT1_IF from the uvm_config_db");

    if (!uvm_config_db #(virtual DUT2_if)::get(this, "","DUT2_IF", my_DUT2_config.DUT2_if_test))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the DUT2_IF from the uvm_config_db");

    if (!uvm_config_db #(virtual DUT3_if)::get(this, "","DUT3_IF", my_DUT3_config.DUT3_if_test))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the DUT3_IF from the uvm_config_db");

    my_DUT1_config.is_active = UVM_ACTIVE;
    my_DUT2_config.is_active = UVM_PASSIVE;
    my_DUT3_config.is_active = UVM_PASSIVE;

    uvm_config_db #(DUT1_config)::set(this, "env_DUT1*", "DUT1_CFG", my_DUT1_config);
    uvm_config_db #(DUT2_config)::set(this, "env_DUT2*", "DUT2_CFG", my_DUT2_config);
    uvm_config_db #(DUT3_config)::set(this, "env_DUT3*", "DUT3_CFG", my_DUT3_config);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
      seq.start(DUT1_env.agt.sqr);
    phase.drop_objection(this);
  endtask
endclass
endpackage
```